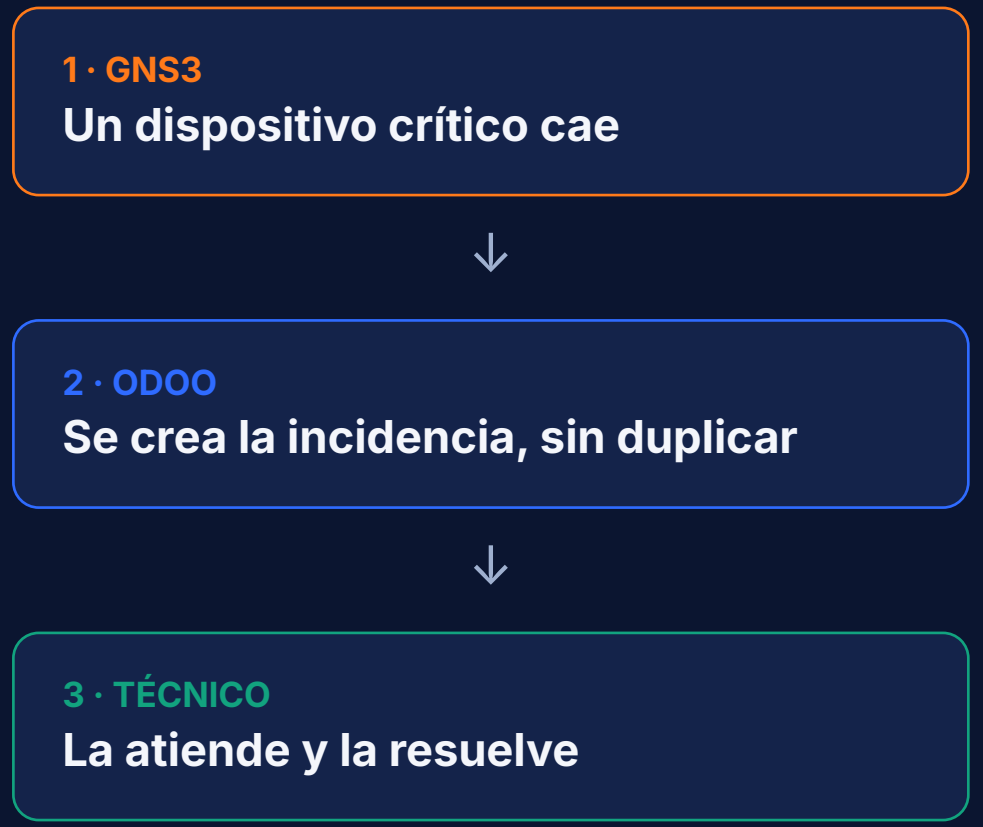
LOW-CODE SOBRE ODOO 17 COMMUNITY

NetPulse

Monitoreo de red con GNS3 y gestión automática de incidencias

Prototipo 1 · Ingeniería de Software II
Universidad Nacional de Colombia · 2026-II

Juliana Parra Caro · Manuel Federico Castro Suarez
Fabian David Mora Martinez · Andres Felipe Arias Gonzalez



FUNDAMENTOS

Qué es Odoo y cómo se construye sobre él

Odoo Community

- Plataforma empresarial **modular**, de código abierto (LGPL) y gratuita
- Ya incluye **login, usuarios y grupos, ORM sobre PostgreSQL, interfaz web y API externa**
- Enterprise (Studio, Helpdesk) es de pago: no lo usamos
- **Low-code no es no-code,** aquí hay Python y XML



Un módulo es una carpeta

__manifest__.py
nombre, versión y dependencias

models/
clases Python = tablas y reglas

security/
grupos, permisos y reglas

views/
XML de listas, kanban y menús

data/
secuencias y tarea programada

ARQUITECTURA

Cómo encajan las piezas

Navegador web
administrador y técnico

Postman / curl
pruebas de la API

push_monitor.py
opcional, Odoo en la nube

Odoo 17 · módulo netpulse

Vistas y menús XML
kanban, lista, formulario, permisos

API externa
POST /jsonrpc

Modelos de dominio
netpulse.device · netpulse.incident

Acción programada
cada minuto → adaptador GNS3

PostgreSQL 15
ORM y SQL

Servidor GNS3
GET /v2/projects/{id}/nodes

Estilo: MVC de Odoo, con puertos y adaptadores solo para la integración con GNS3 · Todo corre en Docker Compose.

CÓDIGO

Estructura de archivos y qué contiene cada uno

Ruta	Qué contiene
__manifest__.py	Metadatos del módulo y lista de archivos que Odoo carga
models/	device.py, incident.py (raíz del agregado) y gns3_adapter.py
security/	Grupos Administrador y Técnico, permisos y regla de registro
views/	Kanban, lista, formulario, gráficos, acciones y menús
data/ · demo/	Secuencia INC-nnnn, tarea cada minuto y datos de ejemplo
tests/	4 pruebas automáticas del módulo
docs/ · postman/	openapi.yaml y colección de 17 pruebas
scripts/	Monitor alterno por API externa
raíz	docker-compose.yml, .env.example, config/odoo.conf, README

DATOS

Base de datos: dónde se crea y dónde vive

Dónde se crea

- Al correr **odoo -d netpulse -i netpulse** Odoo crea la base **netpulse**
- Instala base y mail, y luego el módulo
- El ORM crea **netpulse_device** y **netpulse_incident** desde los modelos Python
- demo.xml carga datos de ejemplo

Dónde vive

- Contenedor **db** (postgres:15)
- Volumen Docker **pgdata**
- Adjuntos en el volumen **odoo-data**
- **down** conserva los datos; **down -v** los borra

Integridad

- unique en nombre y en nodo GNS3
- Dispositivo obligatorio en cada incidencia
- Una incidencia abierta por dispositivo
- Técnico obligatorio desde En proceso

INFRAESTRUCTURA

Contenedores y qué contiene cada uno

db

imagen **postgres:15**

- Motor PostgreSQL con la base netpulse
- Volumen pgdata
- Sin puerto publicado

odoo

imagen **odoo:17.0**

- Servidor Odoo y módulo netpulse
- ./addons montado en /mnt/extra-addons
- odoo.conf y variables GNS3_*
- Puerto 8069 y volumen odoo-data

odoo run --rm

imagen **odoo:17.0**

- Contenedor temporal
- Instala o actualiza el módulo
- Se borra al terminar

Fuera de Docker: GNS3 en tu equipo (host.docker.internal:3080), Postman y, en producción, Caddy o Nginx con HTTPS.

DISEÑO

Patrones implementados

Adapter

PROBLEMA

GNS3 habla su propio JSON (node, started) y el dominio solo entiende activo o caído.

CLASES

gns3.adapter.fetch_nodes → Dispositivo.update_status

State

PROBLEMA

La incidencia permite acciones distintas según su etapa.

CLASES

Tabla de transiciones en write(): Pendiente → En proceso → Resuelta. Saltar una etapa lanza ValidationError.

Observer

PROBLEMA

Avisar al técnico sin acoplar el modelo al canal de aviso. Lo provee el framework.

CLASES

Incidencia crea una actividad pendiente; mail.thread registra cada cambio.

SOLID: cada clase cambia por un motivo · una fuente nueva solo llama a update_status · el cron depende del adaptador, no de HTTP.

DISEÑO

Decisiones de diseño

1. **Odoo como backend**
   seguridad, persistencia y vistas; el dominio queda acoplado al ORM

2. **Adaptador y simulación**
   la demo no depende de la red del laboratorio

3. **Estado Desconocido**
   sin respuesta de GNS3 no se marca Caído: no hay tickets falsos

4. **Una incidencia abierta**
   una caída repetida no satura la bandeja

5. **API JSON-RPC con API keys**
   JWT y REST quedan para el Prototipo 2

6. **Permisos en el servidor**
   write() rechaza lo que la interfaz ya oculta

7. **Monitoreo con cron de Odoo**
   sin servicios extra; script externo para la nube

8. **Metas de calidad**
   seguridad, disponibilidad, modificabilidad y rendimiento

API

Cómo funciona y cómo se escribe en la base de datos

Un solo endpoint: POST /jsonrpc

Cada verbo REST es un método del ORM

GET	**search_read**
POST	**create**
PUT	**write**
DELETE	**unlink**

Ejemplo: crear una incidencia

```
{"jsonrpc": "2.0", "method": "call",
"params": {"service": "object",
"method": "execute_kw", "args": [
"netpulse", 2, "API_KEY",
"netpulse.incident", "create",
[{"device_id": 7, "priority": "2"}], {}]}}
```

→ **{"result": 12}**

PASO 1
uid y API key

PASO 2
permisos y reglas

PASO 3
write() y restricciones

PASO 4
SQL en PostgreSQL

PASO 5
result o error

API

Códigos de respuesta

Respuesta de Odoo	Significado	HTTP equiv.
result	Operación correcta	200 / 201
authenticate → false	Credenciales inválidas en el login	401
AccessDenied	Usuario o API key inválidos	401
AccessError	Sin permiso: grupo, regla de registro o write()	403
ValidationError	Datos inválidos o transición no permitida	400
MissingError	El registro no existe	404
UserError	Regla de negocio, p. ej. iniciar sin técnico	400 / 409

Odoo responde **siempre HTTP 200**: el error va en error.data.name, junto con error.data.debug (traza del servidor), un riesgo documentado.

CALIDAD

Pruebas diseñadas y su objetivo

Postman · 17 peticiones

01–03 Login de admin, técnico e inválido

04–09 CRUD: demuestra que la API funciona

10 API key inválida → AccessDenied

11–12 Técnico elimina o cambia prioridad → AccessError

13 El técnico solo ve su incidencia

14–15 Datos y transición inválidos → ValidationError

Módulo Odoo · 4 pruebas

critical_down crítico caído crea 1 incidencia Alta, sin duplicar

normal_device no crítico no genera ticket

invalid_transition Pendiente → Resuelta falla; el flujo válido guarda fecha de cierre

technician no elimina dispositivos ni reasigna

```
odoo -d netpulse_test -i netpulse --test-enable
```

CIERRE

Seguridad, demo y límites

Seguridad

- Grupos Administrador y Técnico
- Regla: el técnico solo ve lo suyo
- Contraseñas y API keys con hash
- Secretos en .env; list_db = False

Guion de demo

- Simular caída en SW2
- Se crea INC-0001, prioridad Alta
- El admin asigna un técnico
- El técnico inicia y resuelve
- Postman en verde

Límites y siguientes pasos

- Sin JWT ni API REST propia
- HTTPS documentado, sin desplegar
- Errores con data.debug
- Faltan wireframes móviles
- Luego: correo, métricas, auth_jwt

El equipo Scrum

Scrum Master

Felipe Arias

Como SM: facilita los eventos, cuida el tablero y el flujo de ramas y PR, y elimina impedimentos.

Developers

[Juliana Parra· Manuel Castro· Fabian Mora

Estiman, construyen y prueban las historias del sprint; deciden cómo hacer el trabajo.

Backend y dominio

Juliana y Manuel

Modelos de Odoo, reglas de incidencias e integración con GNS3.

HU: 04 · 05 · 06 · 09 · 10 · 11

Lidera: pruebas unitarias y cobertura (Prototipo 2)

API y seguridad

Fabian

Login, token JWT, registro, permisos por rol, OpenAPI y Postman.

HU: 01 · 02 · 07 · 08 · 12

Lidera: CI/CD con GitHub Actions (Prototipo 3)

Cliente y UX

Juliana y Manuel

Vistas de Odoo, dashboard de topología, PWA del técnico y wireframes.

HU: 03 y pantallas de todas las HU

Lidera: pruebas de integración (P2) y Docker (P3)

Revisión cruzada de PR en rotación (1 → 2 → 3 → 1). Liderar un tema no es trabajarlo solo: cualquiera puede tomar tarjetas de otra área.

Metodología: Scrum con tablero Kanban

Scrum da el ritmo: sprints, roles y eventos.

Kanban da la visibilidad del flujo día a día en GitHub Projects.

¿Por qué no Kanban puro? El curso tiene entregas fijas cada pocas semanas, y eso encaja con sprints de tiempo fijo.

Artefactos

Product backlog: 12 HU del MVP (53 pts) más las versiones 2 y 3 del story map

Sprint backlog: lo que el equipo se compromete a hacer en el sprint

Incremento: el prototipo funcional de cada entrega

Métricas

Velocidad: puntos terminados por sprint

Burndown: puntos que faltan cada día del sprint

Un sprint por prototipo

Cada entrega del curso es la Sprint Review de un incremento funcional.

SPRINT 1 · HOY

11 nov

Requisitos, diseño y API segura

SPRINT 2 · 9 días

20 nov

Sistema probado e integrado (cliente + backend)

SPRINT 3 · 19 días

9 dic

Pipeline CI/CD y contenedores

SPRINT 4 · 7 días

16 dic

Entrega final y sustentación

Eventos: Planning al inicio de cada sprint · Daily corta (15 min) · Review = entrega · Retrospectiva después de cada entrega.